

Computer Vision

Tawei Tseng

I. Introduction

The objective of this final project is to implement and understand a basic Incremental SFM system that incrementally reconstructs a sparse 3D point cloud and camera trajectories from multi-view images, thereby gaining deeper insights into multi-view geometry and 3D reconstruction pipelines.

II. Methodology

This project adopts an Incremental Structure from Motion approach. The overall pipeline consists of feature extraction, two-view initialization, 3D reconstruction, and incremental image registration.

A. Feature Detection and Matching

```
K, img_names, init_pair, pixel_th = get_dataset_info(dataset)
```

Figure 1. Features from input images using `get_dataset_info`

First, feature points are detected and described from the input images as shown in figure 1.

```
class FeatureFrontend:
    def __init__(self):
        if hasattr(cv2, "SIFT_create"):
            self.det = cv2.SIFT_create()
            self.norm = cv2.NORM_L2
            self.dtype = np.float32
        else:
            self.det = cv2.ORB_create(5000)
            self.norm = cv2.NORM_HAMMING
            self.dtype = np.uint8
        self.matcher = cv2.BFMatcher(self.norm)

    def detect(self, img_gray):
        kps, desc = self.det.detectAndCompute(img_gray, None)
        if desc is None:
            return [], None
        return kps, desc

    def match_ratio(self, descA, descB, ratio=0.75):
        if descA is None or descB is None:
            return []
        knn = self.matcher.knnMatch(descA, descB, k=2)
        good = []
        for m, n in knn:
            if m.distance < ratio * n.distance:
                good.append(m)
        return good
```

Figure 2. Features extraction and matching class

Figure 2 shows the feature extraction algorithm provided by OpenCV, such as SIFT or ORB, are used to identify keypoints and compute their corresponding descriptors. Feature matching is then performed across image pairs, followed by ratio test and RANSAC to remove outliers and ensure robust correspondences.

B. Two-view Initialization

```
def ransac_E_or_H(x1, x2, epi_th, H_th, iters=2500, seed=1):
    rng = np.random.default_rng(seed)
    N = x1.shape[1]

    best_E, best_E_inl = None, None
    best_H, best_H_inl = None, None

    for _ in range(iters):
        if N >= 8:
            idx = rng.choice(N, size=8, replace=False)
            E = eight_point_E(x1[:, idx], x2[:, idx])
            d = sampson_dist(E, x1, x2)
            inl = d < (epi_th**2)
            if best_E_inl is None or np.sum(inl) > np.sum(best_E_inl):
                best_E, best_E_inl = E, inl

        if N >= 4:
            idx = rng.choice(N, size=4, replace=False)
            H = dlt_homography(x1[:, idx], x2[:, idx])
            H = correct_H_sign(H, x1, x2)
            x2p = H @ x1
            x2p /= x2p[2:3]
            err = np.linalg.norm((x2p[2:] - x2[2:]), axis=0)
            inl = err < H_th
            if best_H_inl is None or np.sum(inl) > np.sum(best_H_inl):
                best_H, best_H_inl = H, inl

    if best_H is not None:
        RTs = homography_to_RT(best_H)
        best = None
        best_count = -1
        inl = best_H_inl

        for k in range(2):
            Rk = RTs[k, :, :3]
            tk = RTs[k, :, 3:4]
            E = skew(tk.ravel()) @ Rk
            R2, t2 = choose_RT_from_E(E, x1[:, inl], x2[:, inl])

            P1 = np.hstack([np.eye(3), np.zeros((3, 1))])
            P2 = np.hstack([R2, t2])
            X4 = triangulate(P1, P2, x1[:, inl], x2[:, inl])
            c = int(np.sum(cheirality_mask(P1, P2, X4)))
            if c > best_count:
                best_count = c
                best = (R2, t2)

    if best is None:
        raise RuntimeError("Homography produced no valid pose.")
    return best[0], best[1], inl
```

Figure 3. A pair of images is recovered by matrix decomposition and cheirality mask to select valid camera pose

Initialization is a crucial step in the SFM pipeline. As shown in figure 3, a pair of initial images is selected to estimate either a homography or an essential matrix based on feature correspondences. The relative camera rotation and translation are recovered through matrix decomposition, and a cheirality check is applied to select the physically valid camera pose in which reconstructed 3D points lie in front of both cameras.

C. Initial 3D Point Cloud Reconstruction

```
P1 = np.hstack([np.eye(3), np.zeros((3, 1))])  
P2 = np.hstack([R12, t12])  
X4 = triangulate(P1, P2, x1[:, inl12], x2[:, inl12])
```

Figure 4. Triangulation for 3D reconstruction from two-view feature points

```
def triangulate(P1, P2, x1, x2):  
    N = x1.shape[1]  
    X = np.zeros((4, N), dtype=float)  
    for i in range(N):  
        x = x1[:, i]  
        xp = x2[:, i]  
        A = np.vstack([  
            x[0] * P1[2] - P1[0],  
            x[1] * P1[2] - P1[1],  
            xp[0] * P2[2] - P2[0],  
            xp[1] * P2[2] - P2[1],  
        ])  
        _, _, Vt = np.linalg.svd(A)  
        Xi = Vt[-1]  
        X[:, i] = Xi / Xi[3]  
    return X # (4,N)
```

Figure 5. Triangulation process class

After obtaining the initial camera poses, triangulation is performed to reconstruct 3D points from corresponding feature points in the two views as shown in figure 4 and figure 5. These reconstructed points form an initial sparse 3D point cloud, which serves as the foundation for subsequent reconstruction steps.

D. Incremental Reconstruction

```
def estimate_t_ransac(x3, X3, R, thresh, iters=3000, seed=0):
    # 2-point RANSAC
    rng = np.random.default_rng(seed)
    N = x3.shape[1]
    if N < 2:
        raise RuntimeError("Need at least 2 2D-3D correspondences.")

    best_inl = None
    best_t = None

    for _ in range(iters):
        idx = rng.choice(N, size=2, replace=False)
        t = estimate_t_from_2d3d_minimal(x3[:, idx], X3[:, idx], R)
        e = reproj_err_norm(x3, X3, R, t)
        inl = e < thresh
        if best_inl is None or np.sum(inl) > np.sum(best_inl):
            best_inl = inl
            best_t = t

    if best_inl is None or np.sum(best_inl) < 6:
        raise RuntimeError("Translation RANSAC failed: too few inliers.")

    # Refit with all inliers
    t_refit = estimate_t_from_2d3d_minimal(x3[:, best_inl], X3[:, best_inl], R)
    return t_refit, best_inl
```

Figure 6. 2D-3D correspondences between image features and existing 3D points are established, and the camera pose is estimated using PnP algorithm with RANSAC

Following initialization, additional images are incrementally added to the reconstruction process. For each new image, 2D–3D correspondences between image features and existing 3D points are established. The camera pose is then estimated using the Perspective-n-Point (PnP) algorithm with RANSAC. New 3D points are triangulated with previously registered images, and the global 3D map is updated accordingly.

III. Results

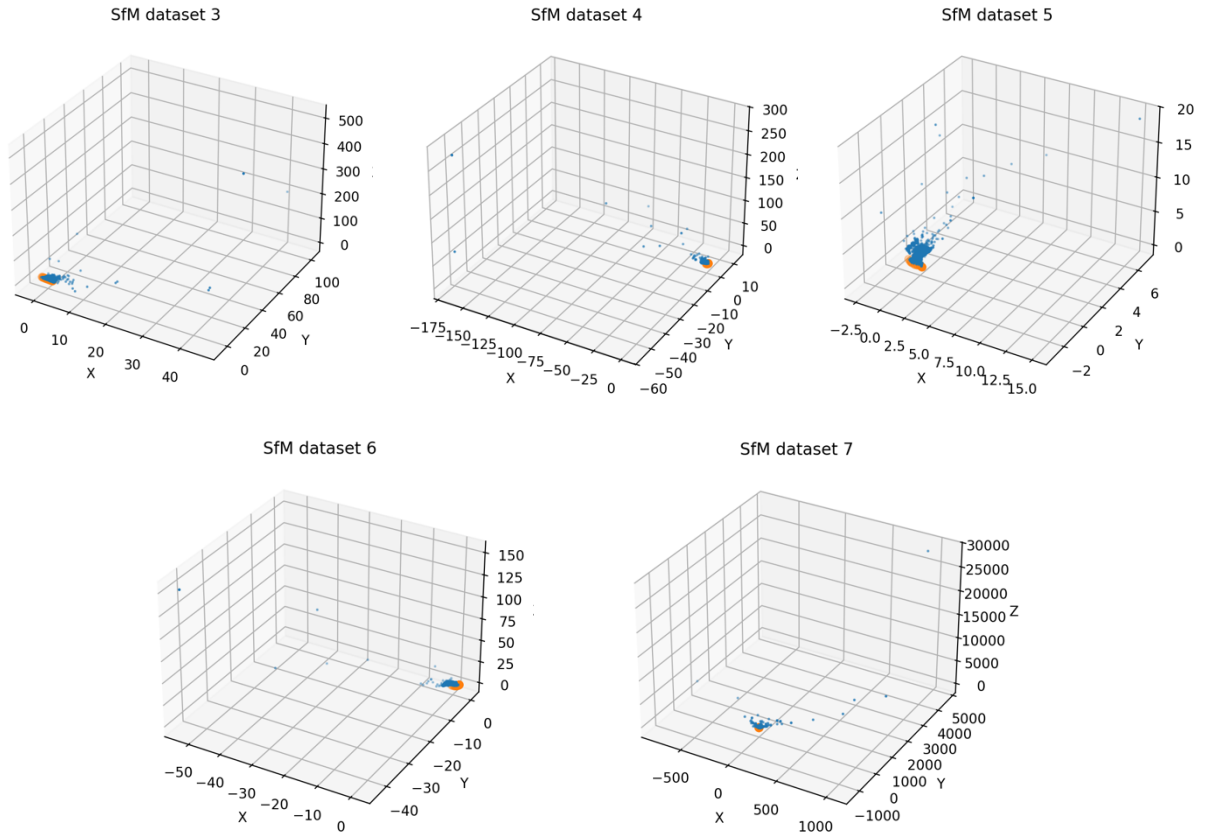


Figure 7. Reconstructing a sparse 3D points cloud from the specific dataset

Using the proposed pipeline, the system successfully estimates camera poses for all input images and reconstructs a sparse 3D point cloud of the scene. The results demonstrate that the Incremental SFM approach effectively recovers both scene geometry and camera motion, validating its applicability to multi-view 3D reconstruction tasks.

IV. Discussion and Limitation

Since global optimization techniques such as Bundle Adjustment are not included, accumulated errors may degrade reconstruction accuracy as the number of images increases. In addition, the quality of feature correspondences and image overlap significantly affects reconstruction performance. Future work may incorporate global optimization and loop closure techniques to improve robustness and accuracy.

V. Conclusion

This final project successfully implements a basic Incremental Structure from Motion system, demonstrating the complete pipeline from feature extraction and camera pose estimation to 3D reconstruction. The project provides valuable insights into the core principles and practical implementation of SFM.